R5A5 — PROGRAMMATION AVANCÉE · SUJET B

Agôn Cup

Dossier de décisions et rapport sur l'usage de l'IA

Denis Zapoi · **Fabio Guerreiro-Marques**

BUT Informatique — IUT de Montreuil, Université Paris 8

Année 2026-2027 · Octobre 2026

SOMMAIRE

Fiche de décision n° 1 — Base de données 2

Fiche de décision n° 2 — Mécanisme de temps réel 3

Fiche de décision n° 3 — Format et destination de la journalisation 5

Rapport — L'usage de l'IA dans le projet 6

FICHE DE DÉCISION N° 1

Base de données

Le problème

Pour commencer, il fallait réfléchir à la question SQL ou NoSQL, et la question a été vite répondue : au vu du projet, le relationnel semblait bien plus adapté. En effet, on a beaucoup de contraintes d'intégrité :

- un joueur dans une seule équipe par tournoi ;
- un seul capitaine par équipe ;
- une seule demande d'adhésion en attente à la fois.

Les options

Ensuite, il fallait trancher entre MySQL et PostgreSQL.

De plus, au départ, on avait envisagé d'utiliser Supabase (qui utilise PostgreSQL). On voulait choisir Supabase parce qu'on pensait, au début, que nous devions déployer le projet en ligne par la suite, et Supabase aurait hébergé la base de données. Cependant, notre professeur nous a dit qu'il allait simplement lancer le projet en local sur sa machine, le front comme le back, pour le regarder. Notre idée de départ, déployer le site avec une base hébergée en ligne, n'avait donc plus lieu d'être. Même si, au final, on n'a pas utilisé Supabase, c'était un argument de plus qui nous a fait pencher vers PostgreSQL.

Ce qu'on a choisi

PostgreSQL, lancé avec Docker.

Un élément nous a fait aller vers PostgreSQL plutôt que MySQL : les index uniques partiels, qui servent par exemple à garantir l'unicité du capitaine. Sans cela, il aurait fallu vérifier ça uniquement au niveau du code.

Ce que ça coûte

Au début, pas forcément de difficulté, puisqu'on codait directement sur les PC de l'IUT, donc tout allait bien. Pour simuler la base de données, on faisait des dumps. Ça a commencé à devenir pénible quand on a vu qu'on allait devoir implémenter une grosse base : il y avait les équipes, les tournois, les jeux, les rôles, les joueurs, les visiteurs, avec en plus des données sur les scores et l'avancée dans le tableau. Nos jeux de données devenaient vraiment énormes. L'idée de passer sur Docker vient de Fabio, et on s'est mis d'accord assez vite avec Denis. Avec le recul, c'était une bonne idée.

L'installation en elle-même nous a pris une dizaine de minutes. Le plus long a été de comprendre pourquoi ça ne marchait pas au départ : on a dû faire nos recherches pour vérifier que, sous Windows, il fallait bien installer WSL et Docker Desktop. On l'a fait sur nos deux ordinateurs.

Un autre coût est venu de Windows. On l'a vu tout de suite, parce que c'est Windows Sécurité qui nous affichait directement les blocages, et ça nous a posé pas mal de problèmes. Son contrôle d'application bloquait les exécutables générés dans l'environnement virtuel Python (comme `pip.exe`), et aussi la bibliothèque native embarquée dans psycopg, le pilote PostgreSQL habituel. Plutôt que de désactiver une protection de Windows, on a changé de pilote pour pg8000, qui est écrit entièrement en Python et n'a donc rien à bloquer. Il est plus lent que psycopg sur de gros volumes, ce qui n'a pas d'importance pour des tournois de quelques équipes, et le changement s'est limité au début de l'adresse de connexion à la base. On l'a noté dans le README pour ne pas bloquer quelqu'un d'autre.

FICHE DE DÉCISION N° 2

Mécanisme de temps réel

Le problème

Le sujet demande que les membres d'une équipe voient les messages arriver sans recharger la page, et que le suivi d'un tournoi se fasse en direct. Le chat est la partie du temps réel qui nous a le plus gênés. Ce n'est pas un chat global : c'est un chat par équipe. Dans un tournoi, il y a plusieurs équipes, et chaque équipe a son propre chat. Il fallait donc que seules les personnes présentes dans une équipe puissent communiquer entre elles, avec un cloisonnement entre les équipes.

Les options

Le cours propose plusieurs mécanismes pour le temps réel. Voici ceux qu'on a regardés, avec les raisons pour lesquelles on les a gardés ou non :

- **Le polling.** On n'a pas essayé : on a vu directement dans le cours que c'était une mauvaise idée, et on l'a donc évité tout de suite. Avoir autant de requêtes qui ne servent à rien, à demander toutes les n secondes « est-ce que tu as reçu un message ? », ne semblait pas approprié au projet.
- **Le SSE** aurait été très approprié, et aurait totalement pu marcher. Pourquoi ne pas l'avoir utilisé ? Contrairement à Socket.IO, il n'y a pas de notion de salon (*room*), donc il aurait fallu écrire nous-mêmes quel utilisateur peut écouter quel tournoi ou quelle équipe.
- **Le WebSocket** : même remarque sur la question des salons.
- **Socket.IO** : les salons sont natifs et la reconnexion est automatique.

Ce qu'on a choisi

Socket.IO, avec la bibliothèque Flask-SocketIO côté back. Les salons ont permis de créer le cloisonnement dont on avait besoin pour le chat : un salon par équipe, et les personnes présentes dans cette équipe peuvent communiquer entre elles. Il propose directement la notion de salon, ce qui allège le travail qu'on doit réaliser.

Ce que ça coûte

Un premier coût, tout bête, a été le test. Pour voir le temps réel, il faut deux comptes différents en même temps. En ouvrant deux pages du même navigateur, la session est partagée : se déconnecter sur une page déconnecte l'autre. On ne le savait pas, et ça nous a bloqués plusieurs jours. Au début, Fabio pensait que le problème venait de son code : qu'il avait mal implémenté quelque chose, ou mal compris une partie du cours. Il a refait certaines parties plusieurs fois, en se disant que c'était peut-être la déconnexion qui posait problème, mais ça ne venait pas de là. La solution était simplement d'utiliser la navigation privée pour avoir deux comptes en parallèle. C'était frustrant de rester bloqué aussi longtemps sur une erreur aussi bête, mais ça fait partie du métier de développeur : on se trompe, et on apprend de ses erreurs.

Un autre coût vient de la reconnexion. Socket.IO se reconnecte automatiquement quand le réseau coupe, mais pas quand c'est le serveur lui-même qui ferme la connexion (par exemple quand la session a expiré) : il considère alors la fermeture comme voulue. Le front a donc dû rouvrir la connexion lui-même dans ce cas.

Le coût le plus important a été la réflexion sur la sécurité. Dès qu'on touche à la sécurité, il faut penser à absolument tout, même à des choses qui ne semblent pas dangereuses au premier abord. Nos bases étaient limitées : on avait fait seulement quelques cours de sécurité pour comprendre le

principe. On n'avait pas d'idée précise de la façon de faire, et on a donc dû beaucoup demander à l'IA, en lui demandant par exemple si une faille pouvait être laissée telle quelle. Elle répondait avec des exemples d'attaques et de conséquences qui nous ont fait corriger rapidement certains points.

Ce qui nous a le plus surpris, c'est une faille dans Werkzeug, la bibliothèque sur laquelle repose Flask (CVE-2026-102598). La semaine d'avant, nos versions de bibliothèques étaient bonnes. Entretemps, une mise à jour de sécurité était sortie et on ne l'avait pas faite. Il a suffi de passer de la version 3.1.8 à la 3.1.9, et notre code n'utilisait pas la fonction concernée, mais on a été très surpris de découvrir une faille dans notre projet alors qu'on n'avait rien changé. Côté front, l'audit de Denis a aussi relevé deux alertes élevées sur Vite, corrigées de la même façon, par une mise à jour.

FICHE DE DÉCISION N° 3

Format et destination de la journalisation

Le problème

On voulait retrouver dans les journaux les traces d'erreurs, le code de réponse de chaque requête, la date, et un message disant si l'opération avait réussi ou non. On voulait aussi y retrouver les refus d'accès et les tentatives d'infraction, par exemple quelqu'un qui essaie d'entrer là où il n'a pas le droit : en gros, tout ce qu'on attend de base de journaux. Chaque requête laisse la route, le code de réponse, la durée, l'utilisateur et un identifiant de requête.

Les options

Le cours cite plusieurs destinations : la console, des fichiers avec rotation, une base de données, des plateformes centralisées et des services cloud.

- **Une base de données.** On a écarté cette solution. La base de l'application a une structure faite pour évoluer, et elle contient des données qu'on veut protéger. Créer une autre base dédiée uniquement à la journalisation aurait été long.
- **Des fichiers.** Il faut les faire tourner, les purger et surveiller le disque, et ils disparaissent avec le conteneur.
- **Une plateforme centralisée ou un service cloud.** C'est lourd à installer et à maintenir, et prévu pour de nombreux services, pas pour une seule application.

Ce qu'on a choisi

La sortie standard, avec une ligne JSON (JSON Lines) par événement. Pour la destination, le cours préconise la console quand l'application tourne dans un conteneur, ce qui est notre cas :

> *« En conteneur, la console redevient la bonne destination. […] L'application se contente de produire du JSON Lines et ignore où il finit. »*

Le niveau suit le résultat : un succès est en INFO, un refus en WARNING, une erreur du serveur en ERROR.

Ce qu'on journalise, et ce qu'on ne journalise pas

On ne journalise ni mot de passe, ni jeton, ni e-mail, ni contenu des requêtes, parce que ce sont des informations personnelles. Même si on a accès au serveur, on n'est pas censés les garder : quelqu'un qui aurait accès à une console verrait aussi les journaux, donc les mots de passe. Au début du développement, on avait mis des mots de passe et des informations de ce genre dans les journaux pour comprendre les erreurs. À la toute fin, on a vérifié qu'on avait tout retiré, et on n'a retrouvé aucun mot de passe oublié.

Ce que ça coûte

Le plus gênant, c'est que les journaux disparaissent quand le conteneur est supprimé. Pour être sûrs que le projet marche pour quelqu'un d'autre et que le README est à jour, on le clonait et on repartait de zéro, comme pour une première installation. On avait alors beaucoup de copies de journaux à garder pour comparer les anciens avec les nouveaux.

L'autre coût est la lecture. Dans d'autres outils, comme Log4j en Java, on peut mettre en page les journaux avec des couleurs. Avec notre pile, on ne savait pas comment faire, et nos premières lignes de JSON étaient difficiles à lire, parce qu'on n'avait pas bien configuré leur mise en page. Le projet a un mode texte (`LOG_FORMAT=texte`) plus lisible en développement.

RAPPORT

L'usage de l'IA dans le projet

1. Ce que nous avons utilisé

On a surtout utilisé Claude, parce que l'un de nous (Fabio) a un abonnement Claude. L'outil principal a été Claude Code, utilisé dans des sessions en ligne reliées à GitHub, qui ouvrait les pull requests. On n'a pas configuré de skills, de sous-agents ni de revue automatisée.

Pour la base de données, on avait proposé plusieurs choix et on a demandé à Claude lequel était le meilleur par rapport à notre projet, en lui donnant le contexte, et s'il existait d'autres technologies. Pour ses audits de sécurité, l'un de nous (Denis) a aussi utilisé Claude.

Tout est visible dans les dépôts : les commits faits avec Claude Code y sont signés. Sur la branche principale, Claude Code a signé 20 commits sur 41 dans le back, et 24 sur 40 dans le front.

Les deux fichiers `CLAUDE.md`. On a des cours de programmation multimédia, qui sont des cours de programmation avec l'IA, et aujourd'hui, si on veut être à jour et rapide, on est obligé d'utiliser l'IA. Quand une personne ouvre notre dépôt avec Claude Code, Claude lit automatiquement `CLAUDE.md` et `API.md` : il a déjà l'architecture, une idée du fonctionnement du projet et les contrats entre le back et le front. Cela permet aux prochaines personnes qui voudront utiliser l'IA sur notre projet d'avoir une IA qui le comprend déjà très bien. Ces fichiers contiennent aussi les règles de travail qu'on a données à Claude : parler français, faire une branche et une pull request par changement, ne jamais fusionner soi-même, ne jamais mettre de secret dans le dépôt, et ne pas rédiger à notre place les fiches de décision ni ce rapport.

2. Nos prompts

Voici trois de nos prompts, recopiés tels quels, avec nos fautes de dictée.

Prompt 1 — partir d'un plan, en validant chaque étape (15 octobre)

> **## Contexte**
> Je travaille sur un projet avec un back-end et un front-end. Pour l'instant, j'ai seulement réalisé :
> - la page de connexion
> - la page principale
>
> J'ai des PR en attente que je n'ai pas encore mergées ; ignore-les pour l'instant.
>
> **## Objectif**
> Je veux avancer fortement sur le projet, en particulier sur :
> - la création de tournois
> - la gestion des tournois
> - le système de chat
> - la gestion des comptes utilisateurs
> - les autres fonctionnalités demandées par le sujet
>
> **## Ce que tu dois faire**
> 1. Lis le sujet du projet et liste les attentes et exigences obligatoires.
> 2. Analyse mon code existant (back et front) : architecture, conventions, ce qui est déjà fait.
> 3. Compare les deux et propose un plan de développement découpé en étapes ordonnées (ex. : étape 1 = comptes, étape 2 = tournois…).
> 4. Réalise uniquement la première étape, puis attends ma validation avant de passer à la suivante.
>
> **## Contraintes**

> - Ne modifie AUCUN fichier toi-même. Je fais les modifications et les commits moi-même.
> - Respecte l'architecture, le style et les conventions déjà présents dans mon code.
> - Pour chaque classe ou fichier concerné, donne le code COMPLET (pas d'extraits ni de « … reste inchangé »).
>
> **## Format de sortie**
> Un document Markdown (.md) par étape, contenant :
> - un résumé de l'étape et de ce qu'elle apporte
> - la liste des fichiers, classée en : ajoutés / modifiés / supprimés
> - pour chaque fichier ajouté ou modifié : son chemin et son code complet
> - pour chaque fichier supprimé : son chemin et la raison
> - les éventuelles migrations, dépendances ou commandes à lancer
> - comment tester que l'étape fonctionne

Ce que ça donnait au départ. À ce moment, on n'avait fait que la page de connexion et la page principale. On voulait que Claude nous dise quelles classes ajouter ou modifier, avec leur code entier, pour faire nous-mêmes les commits.

Ce qu'on a changé. Le prompt demandait d'avancer par petites étapes, validées une à une. Plus tard, on est passés de « je fais les modifications moi-même » à « corrige dans des pull requests » (prompt 3).

Ce que ça a donné. Claude a livré la première étape, les équipes et les demandes d'adhésion côté back : le code complet de 22 fichiers, un plan général en cinq étapes, et 75 nouveaux tests (156 en tout). Il a aussi signalé un défaut : `docker compose up` aurait échoué sur une base neuve, parce que les données de démonstration étaient jouées avant le schéma.

Prompt 2 — cadrer la question pour obtenir une réponse courte (15 octobre)

> Question rapide, sans implémentation :
>
> Dans notre système de chat en temps réel, est-ce que ça vaut le coup d'ajouter l'envoi de photos ou de vidéos ?
>
> Contexte : le projet sera déployé et testé en local par notre prof, sur sa propre machine.
>
> Je veux savoir :
> - si c'est faisable dans ces conditions (notamment ce que ça implique côté serveur)
> - si c'est vraiment nécessaire pour le rendu
>
> Format de réponse attendu : un « oui » ou un « non », suivi de 2 ou 3 explications courtes. Pas de détails techniques ni de code.

Ce que ça donnait au départ. Une idée en passant : permettre d'envoyer des photos ou des vidéos dans le chat.

Ce qu'on a changé. On a précisé qu'on voulait seulement une réponse oui ou non, avec deux ou trois explications, sans partir dans les détails ni commencer à implémenter, pour garder le contrôle de ce qui était fait.

Ce que ça a donné. Claude a répondu non : le sujet demande un chat d'équipe, pas l'envoi de fichiers ; stocker des fichiers demande de vérifier leur type et leur taille et d'appliquer à chacun la même règle d'accès que pour les messages ; et plus il y a de choses à configurer, plus il y a de risques que le projet ne démarre pas chez le professeur. On n'a pas implémenté l'envoi de fichiers.

Prompt 3 — déléguer des corrections dans des pull requests (27 octobre)

> **## Contexte**
> Tu m'as indiqué que la documentation était déjà faite ; je pars de ce principe.
>
> **## Étape 1 : PR de complétion (à faire maintenant)**
> Ouvre une PR sur le back et une PR sur le front pour corriger tout ce qui manque encore. Inclus notamment :
> - l'écran « Mon prochain match » que tu m'as proposé
> - un Docker Compose qui lance toute la pile (back, front, base de données, etc.) — c'est prioritaire
>
> Avant de coder, liste-moi ce qui manque, puis réalise les PR.
>
> **## Étapes suivantes (ne pas commencer sans mon feu vert)**
> 2. Ajustements du site ensemble (textes, contenus, etc.)
> 3. README clairs pour un déploiement local simple sur la machine de mon prof
> 4. Vérification finale de la documentation
> 5. Mise en place de la journalisation
>
> Arrête-toi après l'étape 1 et attends ma validation avant de passer à la suivante.

Ce que ça donnait au départ. Une demande large : corriger ce qui manquait, ajouter l'écran « mon prochain match » et un docker-compose pour toute la pile, puis passer aux README et à la journalisation.

Ce qu'on a changé. Contrairement au prompt 1, on ne faisait plus les modifications nous-mêmes : elles passaient par des pull requests faites par Claude, que l'on relisait et fusionnait ensuite. On a aussi décidé de ne pas envoyer chaque page du cours, mais de laisser Claude s'appuyer sur les chapitres.

Ce que ça a donné. Des pull requests dans les deux dépôts, dont un `docker-compose.complet.yml` pour démarrer la base, l'API et l'interface ensemble, et l'écran « mon prochain match ».

3. Ce que nous avons corrigé

Le code produit par l'IA n'était pas toujours juste, et on a dû le reprendre.

Une course dans l'abonnement à une équipe (Denis)

Lors de l'audit de sécurité de Denis, on a trouvé que la vérification des droits avant d'entrer dans le salon d'une équipe laissait une petite fenêtre : si l'exclusion d'un joueur était validée entre la vérification et l'entrée dans le salon, le joueur exclu entrait quand même et recevait les messages de l'équipe. Un test a été écrit pour reproduire le problème : il échouait avant le correctif. Voici le code livré à la place, qui revérifie les droits après l'entrée :

```
# Revérifié APRÈS l'entrée : une exclusion validée entre la première
# vérification et join_room a retiré le joueur d'un salon où il n'était
# pas encore, puis il y serait entré.
if not espace_service.autoriser_abonnement(id_equipe, identite["id"], identite["ver"]):
    leave_room(salon_equipe(id_equipe))
    metriques.abonnement_refuse()
    return {"ok": False}
```

Le badge « non lu » qui disparaissait au rechargement

En vérifiant le chat à deux comptes, Claude a rejoué le scénario dans un vrai navigateur et a vu qu'après un rechargement de la page, un message arrivé pendant l'absence n'était plus signalé « non lu ». Au rechargement, la session est restaurée après le premier affichage : la messagerie lisait donc ce qui avait déjà été vu avec un identifiant encore vide, et la première lecture des messages écrasait la mémoire enregistrée dans le navigateur. On a écrit un test qui échoue sans le correctif,

puis corrigé ainsi :

```
// Avant : la mémoire est lue une seule fois, au premier affichage.
const [lus, setLus] = useState(() => lireLus(moiId));

// Après : elle est relue quand l'identifiant de l'utilisateur change.
const [memoire, setMemoire] = useState(() => ({ pour: moiId, lus: lireLus(moiId) }));
if (memoire.pour !== moiId) setMemoire({ pour: moiId, lus: lireLus(moiId) });
```

D'autres corrections

- **Le cookie de session** durait un an (valeur par défaut de la bibliothèque), alors que le jeton expire au bout de 12 heures. Ce défaut a été relevé quand on a demandé à Claude de comparer le projet au sujet, puis corrigé en fixant la durée du cookie.
- **Les autres points de l'audit de Denis** : un joueur pouvait modifier le nom de son équipe après la clôture des inscriptions, une écriture venant d'un autre site pouvait être acceptée, et la limite d'inscription était trop large (5 par minute au lieu de 10 par heure).
- **La documentation** comportait des erreurs, trouvées lors d'un audit final : un numéro de version du contrat périmé, et un tableau qui disait le contraire du code sur l'accès des administrateurs aux salons d'équipe.

4. Bilan

Ce que ça nous a apporté. Pour le front, on avait les cours de M. Thomas Louvet, à qui on rend ce projet, donc on avait des bases. Pour le back, c'était différent : Fabio n'avait eu qu'un cours sur Flask et sur les API, et les alternants n'avaient pas eu grand-chose non plus. Denis avait beaucoup plus d'expérience, et Fabio a beaucoup appris de lui sur la façon de coder avec Flask et le fonctionnement d'une API. L'IA l'a aussi aidé en parallèle, en lui expliquant ce qu'il ne comprenait pas, souvent tard le soir.

Ce que ça nous a coûté. Beaucoup d'investissement. On a dû prendre des notes en continu sur ce qu'on faisait, même les choses les plus minimes, pour pouvoir écrire le document que vous avez sous les yeux. Ça a pris du temps, et c'était assez pénible. En plus, les perturbations des dernières semaines nous ont ralentis : avec les manifestations étudiantes autour du CROUS et de l'Université Paris 8, et les blocus, il nous est arrivé de venir à l'IUT en se disant qu'on allait pouvoir travailler, et de ne pas pouvoir étudier. Ce n'étaient pas du tout des conditions favorables à l'apprentissage. Au final, le projet a représenté beaucoup de travail à la maison, sur des heures prévues pour d'autres projets ou d'autres choses.

Ce qu'on ferait autrement. Honnêtement, on ne voit pas vraiment ce qu'on aurait fait autrement. On aurait juste voulu que le projet se déroule dans d'autres conditions.